MicroStitch SDK

MFC Integration Guide

The SDK is designed to sit between your existing microscope camera capture and the MFC display.

1. Copy the SDK into the MFC project

From the provided ZIP, keep:

```
MicroStitchSDK/
|-- include/
|-- lib/
|   `-- MicroStitch.lib
`-- bin/
    `-- MicroStitch.dll
```

For the MFC project:

- Add `MicroStitchSDK/include` to C/C++ > Additional Include Directories
- Add `MicroStitchSDK/lib` to Linker > Additional Library Directories
- Add `MicroStitch.lib` to Linker > Input > Additional Dependencies

At runtime, place:

```
MicroStitch.dll
opencv_world4130.dll
```

in the same directory as the MFC `.exe` (or otherwise make them available through the DLL search path).

The MFC application should use a compatible x64 Release configuration.

2. Include the SDK

In the MFC source:

```
#include <opencv2/opencv.hpp>
#include <microstitch/stitcher.h>
```

Create one persistent Stitcher object:

```
microstitch::Stitcher stitcher;
```

Do not create a new `Stitcher` for every frame. It needs to remain alive while stitching.

3. Pass each microscope frame to the SDK

Where your existing MFC application receives a microscope frame:

```
cv::Mat frame = /* existing microscope frame */;

bool accepted = stitcher.processFrame(frame);
```

The frame should be a normal OpenCV `cv::Mat`.

The SDK handles:

- Feature detection/matching
- Registration
- RANSAC filtering
- Phase-correlation fallback
- Global mosaic positioning
- Mosaic rendering/blending

You don't need to implement any of these in the MFC application.

4. Get the current mosaic

After processing the frame:

```
if (stitcher.hasMosaic())
{
    cv::Mat mosaic = stitcher.getMosaic();

    // Display mosaic in your MFC window
}
```

Your existing MFC/OpenCV display code can then display `mosaic`.

The original microscope frame can continue to be displayed in the first window, while `mosaic` is displayed in the second window.

5. Reset stitching

When the user wants to start a new slide/mosaic:

```
stitcher.reset();
```

The next frame will become the new origin of the mosaic.

6. Typical MFC flow

```
#include <opencv2/opencv.hpp>
#include <microstitch/stitcher.h>

microstitch::Stitcher stitcher;

void OnMicroscopeFrame(const cv::Mat& frame)
{
    // Existing camera display
    DisplayCameraFrame(frame);

    // Process frame
    bool accepted = stitcher.processFrame(frame);

    // Display stitched mosaic
    if (stitcher.hasMosaic())
    {
        cv::Mat mosaic = stitcher.getMosaic();
        DisplayMosaic(mosaic);
    }
}

void OnReset()
{
    stitcher.reset();
}
```

Important
The MFC application owns the camera and the `cv::Mat` frames. The SDK only receives the frames and maintains the stitching state.

The SDK is already built as a Release x64 MSVC DLL, so the MFC application should also be built as x64 Release for the intended integration.